Week 8. Kubernetes Networking Common Issue Troubleshooting & Operations

1. 장애 파악

1-1. 통신 경로를 먼저 나눈다

구분	거치는 구성 요소	먼저 의심할 곳
Pod → Pod	CNI(veth, 라우팅/터널), NetworkPolicy	CNI, 라우팅, MTU, 정책
Pod → Service	DNS, kube-proxy(또는 CNI의 Service 처리), EndpointSlice	DNS, EndpointSlice, kube-proxy
외부 ↔ 클러스터	NodePort / LoadBalancer, 노드 방화벽, SNAT	LB, 노드 라우팅, 외부 방화벽

1-2. 에러 메시지로 구간 추정

보이는 증상	먼저 의심	이유
`connection refused`	목적지 앱 / 포트 / targetPort	목적지까지는 도달했고 아무도 받지 않음
타임아웃 (응답 없음)	정책 drop, 라우팅, 방화벽, MTU	중간에서 패킷이 사라짐
이름 해석 실패 (`NXDOMAIN`, `can't resolve`)	DNS 설정, CoreDNS	IP는 되는데 이름만 안 되는 경우
간헐적 지연 / 타임아웃	DNS(UDP), conntrack, 일부 노드 문제	특정 조건에서만 재현
작은 요청은 되고 큰 응답만 멈춤	MTU	터널 오버헤드 때문에 큰 패킷만 걸림
Pod가 `ContainerCreating`에서 멈춤	CNI, IPAM	Pod에 네트워크를 못 붙임

1-3. 같은 출발지에서 접속 방식을 비교

같은 Pod에서 **Pod IP** → **Service IP** → **이름** 순서로 시도하면 어느 층이 문제인지 좁혀짐.

Pod IP 직접	Service IP	이름(DNS)	의미
실패	실패	실패	앱 또는 CNI/정책 (Pod 간 경로 자체)
성공	실패	실패	Service 쪽 (EndpointSlice, kube-proxy)

Pod IP 직접	Service IP	이름(DNS)	의미
성공	성공	실패	**DNS**
성공	성공	성공	그 구간은 정상, 다른 노드 / 다른 출발지에서 재현 확인

2. 기본 점검 순서

2-1. 순서

1. **Pod / 노드 / 이벤트 확인**: Running인지, 어느 노드인지, 최근 이벤트
2. **직접 호출로 분해**: Pod IP → Service IP → 이름
3. **Service 확인**: Service 존재, selector, **EndpointSlice**가 비어 있지 않은지
4. **DNS 확인**: Pod의 `/etc/resolv.conf`, 클러스터 DNS 질의, CoreDNS 상태
5. **NetworkPolicy 확인**: 해당 Pod를 선택하는 정책, DNS 허용 여부
6. **kube-proxy / CNI 확인**: 데몬 상태와 로그, 노드의 규칙과 라우팅
7. **패킷 확인**: tcpdump, Hubble (SYN은 나가는데 응답이 없는지)

- 공식 Service 디버깅 문서의 순서도 같음: DNS → Service 존재 → IP로 접속 → Service 정의 → **EndpointSlice** → Pod → kube-proxy → (Pod가 자기 Service IP로 못 가는) hairpin 문제

2-2. 명령어 모음

```
# 대상 지정 (값만 바꿔서 사용)
NS=demo
SRC=client
SVC=server
DST_IP=10.244.1.218

# 1. Pod / 노드 / 이벤트
kubectl -n $NS get pod -o wide
kubectl -n $NS get events --sort-by=.lastTimestamp

# 2. 같은 출발지에서 접속 방식별 비교
SVC_IP=$(kubectl -n $NS get svc $SVC -o jsonpath='{.spec.clusterIP}')
kubectl -n $NS exec $SRC -- curl -s -m 3 -o /dev/null -w '%{http_code}\n' http://$DST_IP
```

```
kubectl -n $NS exec $SRC -- curl -s -m 3 -o /dev/null -w '%{h
ttp_code}\n' http://$SVC_IP
kubectl -n $NS exec $SRC -- curl -s -m 3 -o /dev/null -w '%{h
ttp_code}\n' http://$SVC

# 3. Service, EndpointSlice
kubectl -n $NS get svc $SVC -o wide
kubectl -n $NS get endpointslices -l kubernetes.io/service-na
me=$SVC

# 4. DNS
kubectl -n $NS exec $SRC -- cat /etc/resolv.conf
kubectl -n kube-system get pod -l k8s-app=kube-dns
kubectl -n kube-system logs -l k8s-app=kube-dns --tail=30

# 5. NetworkPolicy
kubectl -n $NS get networkpolicy

# 6. kube-proxy
kubectl -n kube-system get pod -l k8s-app=kube-proxy -o wide
```

- DNS 질의 테스트(`nslookup`)는 curl 이미지에 없어서, 공식 문서의 `dnsutils` Pod를 만들어서 `kubectl exec dnsutils -- nslookup kubernetes.default`로 확인
- 위 명령은 **정상일 때도 한 번 실행해서 결과를 저장**해 두면 장애 때 비교 가능

2-3. 노드에서 볼 것

```
# Pod 대역 라우팅과 ARP
ip route
ip neigh

# Service 규칙 (iptables 모드일 때)
sudo iptables-save | grep KUBE-SVC | head

# 규칙 변화 비교 (장애 전/후)
sudo iptables-save > before.txt
sudo iptables-save > after.txt
diff before.txt after.txt
```

```
# 패킷 확인
sudo tcpdump -i any -n host 10.244.1.218 and port 80
```

- 목적지 Pod 쪽에서 tcpdump를 떴는데 **SYN 뒤에 ack가 안 보이면** 수신 Pod 쪽에서 막힌 것 → NetworkPolicy나 iptables 규칙을 의심
- `iptables-save`를 `diff`로 비교하면 두 시점 사이에 Service 규칙이 어떻게 바뀌었는지 확인 가능
- kube-proxy 모드(iptables / ipvs)는 `kube-proxy` ConfigMap의 `mode`에 있고, 바꾸려면 ConfigMap 수정 후 DaemonSet 재시작

3. 구성요소별 장애와 대응

3-1. CNI 장애

증상	원인	확인	대응
노드 `NotReady`, Pod `ContainerCreating`	CNI 설정/바이너리 없음, CNI Pod 미기동	`kubectl get pod -n kube-system`, kubelet 로그	CNI DaemonSet 상태, 설정 파일 복구
새 Pod만 IP를 못 받음	IPAM 고갈, CNI 데몬 오류	CNI Pod 로그, 노드 IP 할당 현황	3-2 참고
업그레이드 후 일부 노드만 이상	노드별 CNI 버전/설정 불일치	노드별 CNI Pod 버전 비교	해당 노드 CNI Pod 재기동, 롤백

3-2. IPAM / IP Pool 문제

확인할 것	내용
노드당 Pod 수	Kubernetes 대규모 클러스터 권장 기준은 노드당 **110 Pod 이하**
노드별 Pod 대역	노드마다 CIDR 블록을 받음
겹침	Pod CIDR, Service CIDR, 노드/VPC 대역이 겹치지 않는지
고갈	노드 대역이 다 차면 새 Pod가 `ContainerCreating`에 머묾

- 증상 예시: 새 Pod만 안 뜸, 기존 Pod는 정상 → IP 할당 쪽 의심
- 대응: IP Pool / 노드별 CIDR 확장, 노드당 Pod 수 제한, 오래된 할당(삭제된 Pod의 IP) 정리
- IPAM 모드와 기본 할당 크기는 CNI마다 다름(Cilium은 cluster-pool이 기본) → 쓰는 CNI 문서에서 확인

3-3. Routing / Overlay / MTU

- **다른 노드의 Pod로만 통신이 안 될 때**: 노드 간 터널 / 라우팅이 문제
  - 노드의 `ip route`에 다른 노드 Pod 대역이 있는지
  - 노드 간 UDP(VXLAN 등) 또는 BGP 포트가 방화벽/보안그룹에서 열려 있는지
- **작은 요청은 되고 큰 응답만 멈출 때**: MTU
  - Calico 공식 문서 기준 오버헤드: IPv4 VXLAN **50바이트**, IP-in-IP 20바이트, IPv4 WireGuard 60바이트
  - 물리 MTU가 1500이면 VXLAN Pod MTU는 1450 → **Pod 인터페이스 MTU가 물리 MTU보다 오버헤드만큼 작아야 함**
  - 확인: Pod 안에서 `ip link`의 mtu, 노드 NIC의 mtu 비교

3-4. kube-proxy / Service

증상	원인	확인	대응
Service 호출 안 됨 (IP도 이름도)	**EndpointSlice가 비어 있음** (selector 불일치, Pod Not Ready)	`get endpointslices`, `describe pod`	label과 selector 맞추기, readiness probe 점검
connection refused	`targetPort`가 컨테이너 포트와 다름	Service와 Pod 정의 비교	targetPort 수정
Pod IP는 되는데 ClusterIP만 안 됨	kube-proxy 미동작, 규칙 미생성	kube-proxy Pod와 로그, `iptables-save`	kube-proxy 재기동, 모드 확인
Pod가 자기 Service IP로만 안 감	hairpin 설정	kubelet `hairpin-mode`	`hairpin-veth` 또는 `promiscuous-bridge`

- `ENDPOINTS`가 `<none>`이면 Service의 `spec.selector`와 Pod의 `metadata.labels`가 일치하는지 먼저 확인
- 라벨이 맞는데도 비어 있으면 Pod가 Ready가 아닌지(readiness probe) 확인
- kube-proxy 로그에 `Setting endpoints for ...` 줄이 보여야 정상

3-5. DNS

확인	정상 기준
Pod의 `/etc/resolv.conf`	`nameserver`가 kube-dns Service IP, `search`에 `<ns>.svc.cluster.local` 등, `options ndots:5`
`nslookup kubernetes.default`	이름과 IP가 나옴
CoreDNS Pod	`k8s-app=kube-dns` 라벨, Running/Ready
CoreDNS 로그	오류, `loop` 감지 메시지 없음

- **ndots:5**: 점이 5개 미만인 이름은 search 도메인을 먼저 붙여서 조회 → 외부 도메인 조회가 여러 번 실패한 뒤 성공해서 느려질 수 있음 (끝에 `.`를 붙이면 건너뜀)
- **systemd-resolved 루프**: 노드의 `/etc/resolv.conf`가 `127.0.0.53` 스텁이면 CoreDNS가 자기 자신에게 질의하는 루프가 생길 수 있음 → kubelet `-resolv-conf`로 해결
- **hostNetwork Pod**는 `dnsPolicy: ClusterFirstWithHostNet`을 줘야 클러스터 DNS를 씀
- `/etc/resolv.conf`가 **DNS 문제의 첫 확인 지점**, CoreDNS 기본 `cache 30` 때문에 Pod가 되살아나도 몇 초간 이름이 안 풀릴 수 있음

3-6. NetworkPolicy

- 기본은 **허용**: 정책이 없으면 ingress/egress 모두 열려 있음
- 정책이 Pod를 **선택하는 순간** 그 방향은 정책이 허용한 것만 통과
- **default-deny egress를 걸면 DNS도 막힘** → DNS(53) 허용 정책을 같이 넣어야 함
- 허용된 연결의 **응답 트래픽은 자동 허용**

증상	확인
정책 적용 후 전부 끊김	egress deny로 DNS까지 막혔는지
정책을 만들었는데 적용 안 됨	podSelector 라벨, namespace, CNI가 정책을 지원하는지
특정 Pod만 막힘	그 Pod에 걸린 모든 정책(합집합으로 허용)

3-7. External

증상	원인	확인
Pod → 외부 안 됨	SNAT(masquerade) 누락, 노드 기본 라우팅, 외부 방화벽/DNS	노드에서 직접 외부 접속 가능한지, egress 정책
LoadBalancer `Pending`	LB를 만들어 줄 컨트롤러(클라우드/MetalLB 등)가 없음	Service 이벤트, 컨트롤러 Pod
특정 노드로 들어온 요청만 실패	해당 노드의 NodePort 규칙, 방화벽, 백엔드 위치	노드별로 NodePort 호출 비교

4. 운영 중 자주 터지는 케이스

케이스	증상	원인	확인	대응 / 예방
정책 적용 후 장애	갑자기 타임아웃	egress deny가 DNS 차단	Hubble DROPPED, 정책 목록	DNS 허용 정책 기본 포함, audit 모드로 먼저 점검

케이스	증상	원인	확인	대응 / 예방
EndpointSlice 비어 있음	Service만 안 됨	label/selector 불일치, readiness 실패	`get endpointslices`	배포 매니페스트 점검, probe 점검
노드 하나만 이상	그 노드 Pod만 통신 불가	해당 노드 CNI/kube-proxy/방화벽	노드별 비교	해당 노드 데몬 재기동, 노드 교체
큰 응답만 멈춤	로그인은 되는데 다운로드 실패	MTU 불일치	`ip link` mtu 비교	Pod MTU를 물리 MTU − 오버헤드로 설정
새 Pod만 안 뜸	`ContainerCreating`	IP 고갈	CNI 로그, 노드 CIDR 사용량	Pool 확장, 노드당 Pod 수 관리
DNS 간헐 지연	가끔 느리거나 실패	ndots, CoreDNS 부하, conntrack	CoreDNS 로그, `resolv.conf`	FQDN 사용, NodeLocal DNS 검토
CNI 업그레이드 후	일부 노드만 이상	노드별 버전 불일치	CNI Pod 버전	단계별 롤아웃, 롤백 계획

5. Cilium 환경에서 확인

- **Hubble**이 "어디까지 갔는지"를 바로 보여 줌

보이는 것	의미
`DROPPED` + `Policy denied`	NetworkPolicy가 막음 (실습: `policy-verdict:none INGRESS DENIED`)
`FORWARDED`인데 응답이 없음	목적지 앱 / 반대 방향 flow 확인
flow가 아예 없음	다른 노드(Relay 필요), ring buffer 덮어쓰기, 데이터패스를 안 거침

- Week 7 실습에서 NetworkPolicy 후 intruder의 SYN이 1초 간격으로 재전송되며 `DROPPED`로 보인 것이 **타임아웃**
- 이 경우 목적지 앱 문제가 아니라 **정책 drop**이라는 걸 Hubble이 바로 구분해 줌

결론

> 장애는 **경로(Pod / Service / External)부터 나누고**, 같은 출발지에서 **Pod IP → Service IP → 이름** 순으로 좁혀 간다

Q1. 장애를 어디서부터 분해해서 볼지

- 통신 경로를 Pod→Pod / Pod→Service / 외부로 나누고, 범위(전체 / 특정 노드 / 특정 Service)를 먼저 확인

- 에러 메시지로 구간 추정: refused는 목적지 앱, 타임아웃은 중간 drop, 이름 해석 실패는 DNS, 큰 응답만 멈추면 MTU
- 같은 Pod에서 Pod IP / Service IP / 이름 결과 조합으로 어느 층인지 판단
- 변경 이력과 정상 기준선을 같이 기록

Q2. 네트워크 문제를 확인하는 기본 점검 순서

- Pod·노드·이벤트 → 직접 호출 분해 → Service·EndpointSlice → DNS → NetworkPolicy → kube-proxy·CNI → 패킷(tcpdump, Hubble)
- Service 문제는 **EndpointSlice**를 먼저 확인
- 노드 단위는 `ip route`, `iptables-save`(diff), tcpdump로 확인

Q3. 운영 중 자주 터지는 케이스와 대응 방법

- 정책: egress deny 시 DNS 허용 필수, 적용 전 audit 모드로 점검
- IPAM: 노드당 Pod 수와 CIDR 고갈 관리
- MTU: Pod MTU = 물리 MTU – 오버헤드(VXLAN 50바이트)
- DNS: `ndots:5`, systemd-resolved 루프, hostNetwork의 dnsPolicy
- 노드 한 대만 이상하면 해당 노드의 CNI / kube-proxy / 방화벽부터 비교

질문

공식 문서에서 `ndots:5` 때문에 외부 도메인을 조회할 때 search 도메인 조회가 여러 번 먼저 일어나 느려질 수 있다고 하고, NodeLocal DNSCache가 UDP 타임아웃을 줄여 준다고 되어 있음. 실무에서는 DNS가 간헐적으로 느릴 때 앱의 `ndots` 설정이나 FQDN 사용부터 조정하는지, 아니면 NodeLocal DNSCache 같은 클러스터 수준 대응을 먼저 도입하는지 궁금함.